

Tugas 3

Karirnex – Data Analyst

Karena keterangan sudah ditulis di dalam colab maka yang ditulis di word hanya keterangan tambahan

1. Soal ditampilkan di bawah. Untuk jawaban akan ditampilkan sekaligus baik kodingan maupun Output karena untuk soal ini terbilang cukup sederhana.

Soal 1 :

Tampilkan:

1. Jumlah baris dan jumlah kolom dari data (df)
2. Nama-nama semua kolom yang ada
3. 10 baris pertama dari data

Code

```
# 1. Tampilkan jumlah baris dan jumlah kolom dari df
# ada di atas xixixi
print("Jumlah baris dan kolom:", df.shape)

# 2. Tampilkan nama-nama semua kolom
# \n digunakan sebagai enter
# hanya dapat digunakan dalam bentuk string
print("\nNama-nama kolom:\n", df.columns)

# 3. Tampilkan 10 baris pertama dari data
# print kosong dibuat sebagai pemisah hasil print sebelumnya dengan hasil dibawah
# singkatnya print kosong = enter sama seperti \n tapi diluar string
print()
df.head(10)
```

Output nomor 1 dan 2

```
Jumlah baris dan kolom: (10000, 16)

Nama-nama kolom:
Index(['sales_date', 'order_id', 'customer_name', 'branch', 'product_name',
      'category', 'color', 'price', 'quantity', 'payment_type', 'trade_in',
      'discount', 'total', 'total_sales', 'status', 'branch_address'],
      dtype='object')
```

Karena Output nomor 3 terlalu panjang, saya hanya dapat memberikan potongan dari outputnya

	sales_date	order_id	customer_name	branch	product_name	category	color	price	quantity
0	2025-12-13	TRX00001	Customer_913	Jakarta	Daihatsu Xenia 1.5 R CVT	MPV	Silver Metalik	240000000	1
1	2025-10-15	TRX00002	Customer_261	Jakarta	Toyota Rush 1.5 S GR Sport CVT	SUV	Hitam Metalik	300000000	1
2	2025-01-04	TRX00003	Customer_4465	Kendari	Honda BR-V 1.5 E CVT	MPV	Merah	310000000	1
3	2025-02-17	TRX00004	Customer_2788	Padang	Honda Brio Satya E CVT	Hatchback	Merah	190000000	1
4	2025-10-02	TRX00005	Customer_2941	Karawang	Isuzu D-Max Double Cabin 4x4 AT	Pickup	Hitam Metalik	520000000	1
5	2025-07-05	TRX00006	Customer_3101	Bandung	Wuling Air ev Long Range	City Car (EV)	Abu-abu	250000000	1
6	2025-02-21	TRX00007	Customer_570	Jakarta	Honda BR-V 1.5 E CVT	MPV	Hitam Metalik	310000000	2
7	2025-12-16	TRX00008	Customer_2989	Pontianak	Mitsubishi Pajero Sport Dakar 4x2 AT	SUV	Putih Pearl	580000000	1

2. Soal ditampilkan di bawah.

Soal 2: Jumlah Transaksi per Kategori Mobil

Hitung ada berapa banyak transaksi untuk masing-masing kategori mobil (kolom `category`), diurutkan dari yang paling banyak.

Bonus (opsional): buat bar chart dari hasilnya.

Code

```
# Hitung jumlah transaksi per kategori mobil (category), urutkan dari yang paling banyak
transaksi_kategori = df['category'].value_counts()
print(transaksi_kategori, '\n')

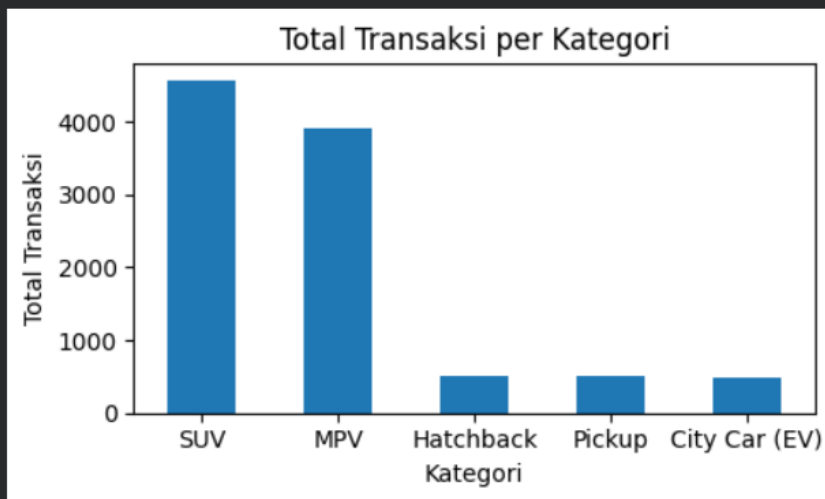
# (optional) buat bar chart dari hasilnya
# Untuk membuat chart dibutuhkan library matplotlib.pyplot
import matplotlib.pyplot as plt

# plot digunakan untuk memanggil library matplotlib.pyplot
# kind digunakan untuk menentukan jenis chart yang digunakan
# figsize digunakan untuk menentukan ukuran chart (lebar, tinggi)
# title digunakan untuk memberikan judul pada chart
# x dan y label digunakan untuk title pada sumbu x dan y
# xticks digunakan agar label pada sumbu x tidak muncul secara vertikal
# tight_layout digunakan untuk mengatur jarak antara chart dan label

transaksi_kategori.plot(kind="bar", figsize=(5, 3), title="Total Transaksi per Kategori")
plt.ylabel("Total Transaksi")
plt.xlabel("Kategori")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

Output

```
category
SUV          4565
MPV          3915
Hatchback     519
Pickup        516
City Car (EV)  485
Name: count, dtype: int64
```



3. Soal ditampilkan di bawah.

Soal 3: Cabang dengan Transaksi Terbanyak

Dari semua cabang yang ada, cabang mana yang punya **jumlah transaksi terbanyak** (bukan pendapatan, tapi jumlah baris transaksinya)? Tampilkan juga angkanya.

Code dan Output

```
# Hitung jumlah transaksi per cabang (branch), urutkan dari yang paling banyak
# lalu tampilkan cabang dengan jumlah transaksi terbanyak

# value_counts digunakan untuk menghitung jumlah kemunculan setiap nilai unik dalam sebuah kolom

jumlah_transaksi_per_cabang = df['branch'].value_counts()
print(jumlah_transaksi_per_cabang, '\n')
```

```
branch
Jakarta      1058
Padang        1044
Malang        1044
Bandung       989
Batam         986
Pontianak     982
Palembang     981
Kendari       976
Karawang      973
Semarang      967
Name: count, dtype: int64
```

4. Soal ditampilkan di bawah. Karena soal cukup rumit maka kodingan dan Output ditampilkan secara terpisah.

Soal 4: Bersihkan dan Siapkan Data

Sebelum data ini dipakai buat analisis lebih lanjut, kamu diminta membersihkan dan merapikannya dulu. Lakukan langkah-langkah berikut secara berurutan, dan simpan hasilnya ke `df` (menimpa data yang lama):

1. Cek apakah ada **missing value** di tiap kolom. Kalau kolom `trade_in` ada yang kosong, isi dengan `0`.
2. Cek apakah ada **baris duplikat** (baik duplikat penuh maupun duplikat `order_id`). Kalau ada, hapus baris duplikatnya.
3. Standarisasi kolom `payment_type` : hilangkan spasi berlebih di depan/belakang, dan samakan kapitalisasinya jadi Title Case (contoh: `"kredit"` jadi `"Kredit"`).
4. Ubah kolom `sales_date` yang masih berupa teks jadi tipe data tanggal (`datetime`).
5. Setelah semua langkah di atas selesai, tampilkan `df.info()` buat membuktikan datanya sudah lebih rapi (nggak ada missing value yang bermasalah, tipe data `sales_date` sudah benar).

1. Code dan Output Sebelum Cleaning

```
# # 1. Cek missing value, lalu isi trade_in yang kosong dengan 0
# df["trade_in"] = df["trade_in"].fillna(0)

# Cek data trade_in
# dilakukan sebelum cleaning untuk memeriksa apakah ada data kosong
# dan dilakukan setelah cleaning untuk memastikan data sudah terisi
df['trade_in'].head(5)
```

	trade_in
0	74000000
1	0
2	111000000
3	0
4	93000000

dtype: int64

Dikarenakan data sudah bersih. Output setelah cleaning tetap sama

```
# # 1. Cek missing value, lalu isi trade_in yang kosong dengan 0
df["trade_in"] = df["trade_in"].fillna(0)

# Cek data trade_in
# dilakukan sebelum cleaning untuk memeriksa apakah ada data kosong
# dan dilakukan setelah cleaning untuk memastikan data sudah terisi
df['trade_in'].head(5)
```

	trade_in
0	74000000
1	0
2	111000000
3	0
4	93000000

dtype: int64

2. Code dan Output sebelum cleaning

```
# # 2. Cek dan hapus baris duplikat (baik duplikat penuh maupun duplikat order_id)
# df = df.drop_duplicates()

# Cek data trade_in
# dilakukan sebelum cleaning untuk memeriksa apakah ada data duplikat
# pengecekan dilakukan dengan memeriksa jumlah data
print("Jumlah baris sebelum hapus duplikat:", len(df))

# cek data dilakukan kembali setelah cleaning untuk memastikan data sudah bersih
# dilakukan dengan memeriksa apakah jumlah data berkurang
# jika tidak berkurang berarti data sudah bersih dari awal
# print("Jumlah baris setelah hapus duplikat:", len(df))

Jumlah baris sebelum hapus duplikat: 10000
```

Dikarenakan data sudah bersih. Output setelah cleaning tetap sama

```
# # 2. Cek dan hapus baris duplikat (baik duplikat penuh maupun duplikat order_id)
df = df.drop_duplicates()

# Cek data trade_in
# dilakukan sebelum cleaning untuk memeriksa apakah ada data duplikat
# pengecekan dilakukan dengan memeriksa jumlah data
# print("Jumlah baris sebelum hapus duplikat:", len(df))

# cek data dilakukan kembali setelah cleaning untuk memastikan data sudah bersih
# dilakukan dengan memeriksa apakah jumlah data berkurang
# jika tidak berkurang berarti data sudah bersih dari awal
print("Jumlah baris setelah hapus duplikat:", len(df))

Jumlah baris setelah hapus duplikat: 10000
```

3. Code dan Output sebelum preparation

```
# # 3. Standarisasi kolom payment_type (strip + title case)
# Cek isi sebelum standarisasi
df['payment_type'].unique()

# df['payment_type'] = df['payment_type'].str.strip().str.title()
# # Cek isi setelah standarisasi
# df['payment_type'].unique()

array(['Cash + Trade In', 'Kredit', 'Kredit + Trade In', 'Cash'],
      dtype=object)
```

Output setelah preparation sama

```
# # 3. Standarisasi kolom payment_type (strip + title case)
# Cek isi sebelum standarisasi
# df['payment_type'].unique()

df['payment_type'] = df['payment_type'].str.strip().str.title()
# Cek isi setelah standarisasi
df['payment_type'].unique()

array(['Cash + Trade In', 'Kredit', 'Kredit + Trade In', 'Cash'],
      dtype=object)
```

4. Code dan Output Sebelum preparation

```
# # 4. Ubah kolom sales_date jadi tipe data datetime
# Cek sebelum ubah tipe data
print(df["sales_date"].dtype)
df["sales_date"].head()

# Mengubah tipe data tanggal menjadi datetime menggunakan pandas
# error='coerce' digunakan untuk mengubah nilai yang tidak dapat diubah
# menjadi datetime menjadi NaT (Not a Time)
# penggunaan coerce opsional. hanya sebagai antisipasi jika terdapat data yang tidak valid
# tanpa coerce akan terjadi error. dengan coerce perintah akan terus berjalan tanpa error

# df["sales_date"] = pd.to_datetime(df["sales_date"], errors="coerce")

# print(df["sales_date"].dtype)
# df["sales_date"].head()
```

object

sales_date

0 2025-12-13

1 2025-10-15

2 2025-01-04

3 2025-02-17

4 2025-10-02

dtype: object

Output setelah preparation

```
# # 4. Ubah kolom sales_date jadi tipe data datetime
# Cek sebelum ubah tipe data
# print(df["sales_date"].dtype)
# df["sales_date"].head()

# Mengubah tipe data tanggal menjadi datetime menggunakan pandas
# error='coerce' digunakan untuk mengubah nilai yang tidak dapat diubah
# menjadi datetime menjadi NaT (Not a Time)
# penggunaan coerce opsional. hanya sebagai antisipasi jika terdapat data yang tidak valid
# tanpa coerce akan terjadi error. dengan coerce perintah akan terus berjalan tanpa error

df["sales_date"] = pd.to_datetime(df["sales_date"], errors="coerce")

print(df["sales_date"].dtype)
df["sales_date"].head()
```

datetime64[ns]

	sales_date
0	2025-12-13
1	2025-10-15
2	2025-01-04
3	2025-02-17
4	2025-10-02

dtype: datetime64[ns]

5. Code dan Output

```
## 5. Tampilkan df.info() untuk membuktikan data sudah lebih rapi
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   sales_date            10000 non-null  datetime64[ns]
1   order_id              10000 non-null  object  
2   customer_name         10000 non-null  object  
3   branch                10000 non-null  object  
4   product_name          10000 non-null  object  
5   category              10000 non-null  object  
6   color                 10000 non-null  object  
7   price                 10000 non-null  int64   
8   quantity              10000 non-null  int64   
9   payment_type          10000 non-null  object  
10  trade_in              10000 non-null  int64   
11  discount              10000 non-null  int64   
12  total                 10000 non-null  int64   
13  total_sales           10000 non-null  int64   
14  status                10000 non-null  object  
15  branch_address        10000 non-null  object  
dtypes: datetime64[ns](1), int64(6), object(9)
memory usage: 1.2+ MB
```

5. Soal ditampilkan di bawah

Soal 5: Kombinasi Cabang & Kategori dengan Pendapatan Tertinggi

Manajemen mau tahu: **kombinasi cabang dan kategori mobil apa** yang menghasilkan pendapatan (`total_sales`) paling besar? Ini penting buat mereka menentukan cabang mana yang perlu ditambah stok kategori tertentu.

Kerjakan langkah-langkah berikut:

1. Ambil hanya transaksi dengan status `"completed"` (transaksi yang batal/refund tidak dihitung sebagai pendapatan).
2. Kelompokkan data berdasarkan **kombinasi** `branch` dan `category` sekaligus (bukan salah satu aja), lalu jumlahkan `total_sales`-nya.
 - *Petunjuk: `groupby()` bisa menerima lebih dari satu kolom sekaligus, dengan cara `df.groupby(["kolom1", "kolom2"])`.*
3. Urutkan hasilnya dari pendapatan terbesar, lalu tampilkan **5 kombinasi cabang + kategori** dengan pendapatan tertinggi.
4. Buat bar chart horizontal dari 5 kombinasi teratas tersebut, biar gampang dipresentasikan.

Tantangan tambahan (opsional, buat yang mau nyoba lebih jauh): dari 5 kombinasi teratas itu, hitung juga berapa **rata-rata harga per unit** (`price`) untuk masing-masing kombinasi tersebut.

1. Code dan Output

```
# # 1. Filter transaksi yang statusnya "completed"
```

```
transaksi_selesai = df[df["status"] == "completed"]  
print("Jumlah transaksi completed:", len(transaksi_selesai))  
transaksi_selesai.head(5)
```

color	price	quantity	payment_type	trade_in	discount	total	total_sales	status	branch_address
Silver Metalik	240000000	1	Cash + Trade In	74000000	0	240000000	166000000	completed	Jl. MT. Haryono No. 12, Jakarta
Hitam Metalik	300000000	1	Kredit	0	0	300000000	300000000	completed	Jl. MT. Haryono No. 12, Jakarta
Merah	310000000	1	Kredit + Trade In	111000000	0	310000000	199000000	completed	Jl. Ahmad Yani No. 33, Kendari
Merah	190000000	1	Kredit	0	0	190000000	190000000	completed	Jl. Khatib Sulaiman No. 10, Padang
Abu-abu	250000000	1	Kredit	0	0	250000000	250000000	completed	Jl. Soekarno Hatta No. 88, Bandung

2. Code 1

```
# # 2. Groupby kombinasi branch dan category, jumlahkan total_sales

# Reset index agar branch bisa di grouping secara terpisah
# () digunakan setelah = agar kode bisa ditulis dengan baris jadi lebih mudah dibaca
penjualan_per_kombinasi = (
    df.groupby(["branch", "category"])["total_sales"]
    .sum()
    .reset_index()
)

# Mengambil index branch agar dapat di grouping/branch
urutan_branch = (
    penjualan_per_kombinasi.groupby("branch")["total_sales"]
    .sum()
    .sort_values(ascending=False)
    .index
)

# Menyimpan branch sebagai urutan
# Categorical digunakan agar data yang telah disusun dapat disimpan dan digunakan kembali
penjualan_per_kombinasi["branch"] = pd.Categorical(
    penjualan_per_kombinasi["branch"],
    # Mengambil data yang telah disusun
    categories = urutan_branch,
    # Menyimpan data agar dapat digunakan kembali
    ordered = True
)

# Mengurutkan data berdasarkan total_sales
```

Code 2

```
# Mengurutkan data berdasarkan total_sales
penjualan_per_kombinasi = (
    penjualan_per_kombinasi
    .sort_values(["branch", "total_sales"], ascending = [True, False])
)

# Menunjukkan hasil branch, category, dan total_sales
print(penjualan_per_kombinasi)
# Hasil diurutkan dari branch dengan hasil tertinggi
# Kemudian diurutkan berdasarkan category dengan hasil tertinggi di tiap branch
```

Output hanya diberikan potongan karena terlalu banyak

	branch	category	total_sales
29	Malang	SUV	169456000000
27	Malang	MPV	110368000000
28	Malang	Pickup	27475000000
25	Malang	City Car (EV)	10032000000
26	Malang	Hatchback	7336000000
14	Jakarta	SUV	172403000000
12	Jakarta	MPV	103477000000
13	Jakarta	Pickup	26885000000
10	Jakarta	City Car (EV)	11925000000
11	Jakarta	Hatchback	6516000000
34	Padang	SUV	170755000000
32	Padang	MPV	108692000000
33	Padang	Pickup	20002000000
31	Padang	Hatchback	11288000000
30	Padang	City Car (EV)	9826000000
39	Palembang	SUV	177343000000
37	Palembang	MPV	96344000000
38	Palembang	Pickup	24338000000
36	Palembang	Hatchback	8209000000
35	Palembang	City Car (EV)	7190000000
4	Bandung	SUV	166898000000
2	Bandung	MPV	97624000000
3	Bandung	Pickup	25304000000
0	Bandung	City Car (EV)	12386000000
1	Bandung	Hatchback	7068000000
9	Batam	SUV	167034000000
7	Batam	MPV	93843000000
8	Batam	Pickup	27980000000
5	Batam	City Car (EV)	10813000000
6	Batam	Hatchback	7354000000
24	Kendari	SUV	165997000000
22	Kendari	MPV	93548000000
23	Kendari	Pickup	30209000000
20	Kendari	City Car (EV)	7654000000

Index yang disebelah kiri adalah index berdasarkan susunan nama branch.

Oleh karena itu, bandung dan batam berada pada index paling atas (0-9).

3. Karena terdapat sedikit kesalah pahaman terhadap soal, saya menuliskan 3 cara pengerjaan. Code saya tampilkan dibawah

```
# # 3. Urutkan dari terbesar, ambil 5 kombinasi teratas

# # Terbesar apa? per category? per total_sales? per branch?
# # karena bingung dan bisa jadi saya tampilkan ketiganya dengan cara yang serupa

# Dilakukan grouping terlebih dahulu
# Jika kolom yang digunakan lebih dari satu, tambah [] di dalam groupby
# Total sales dijumlahkan dan diurutkan dari yang terbesar

# agar tidak mengganggu kode pada nomor 2, saya membuat variabel baru

# total_sales terbesar
total_sales_tertinggi = (
    df.groupby(["branch", "category"])["total_sales"]
      .sum().sort_values(ascending=False)
)
total_sales_tertinggi.head(5)

# branch terbesar
branch_tertinggi = (
    df.groupby("branch")["total_sales"]
      .sum().sort_values(ascending=False)
)
branch_tertinggi.head(5)

# category terbesar
category_tertinggi = (
    df.groupby("category")["total_sales"]
      .sum().sort_values(ascending=False)
)
category_tertinggi.head(5)

# # Jika dilihat kembali yang dibutuhkan memang hanya branch & category
# # jadi seharusnya masing-masing branch dan category tidak perlu
```

```
# # 3. Urutkan dari terbesar, ambil 5 kombinasi teratas

# # Terbesar apa? per category? per total_sales? per branch?
# # karena bingung dan bisa jadi saya tampilkan ketiganya dengan cara yang serupa

# Dilakukan grouping terlebih dahulu
# Jika kolom yang digunakan lebih dari satu, tambah [] di dalam groupby
# Total sales dijumlahkan dan diurutkan dari yang terbesar

# agar tidak mengganggu kode pada nomor 2, saya membuat variabel baru

# total_sales terbesar
total_sales_tertinggi = (
    df.groupby(["branch", "category"])["total_sales"]
    .sum().sort_values(ascending=False)
)
total_sales_tertinggi.head(5)

# # branch terbesar
# branch_tertinggi = (
#     df.groupby("branch")["total_sales"]
#     .sum().sort_values(ascending=False)
# )
# branch_tertinggi.head(5)

# # category terbesar
# category_tertinggi = (
#     df.groupby("category")["total_sales"]
#     .sum().sort_values(ascending=False)
# )
# category_tertinggi.head(5)

# # Jika dilihat kembali yang dibutuhkan memang hanya branch & category
# # jadi seharusnya masing-masing branch dan category tidak perlu
```

Karena data yang dibutuhkan data kombinasi maka saya hanya menampilkan satu output.

		total_sales
branch	category	
Palembang	SUV	177343000000
Jakarta	SUV	172403000000
Padang	SUV	170755000000
Malang	SUV	169456000000
Batam	SUV	167034000000

dtype: int64

4. Code untuk menampilkan 3 chart berdasarkan 3 data berbeda.

```
# 4. Buat bar chart horizontal dari 5 kombinasi teratas

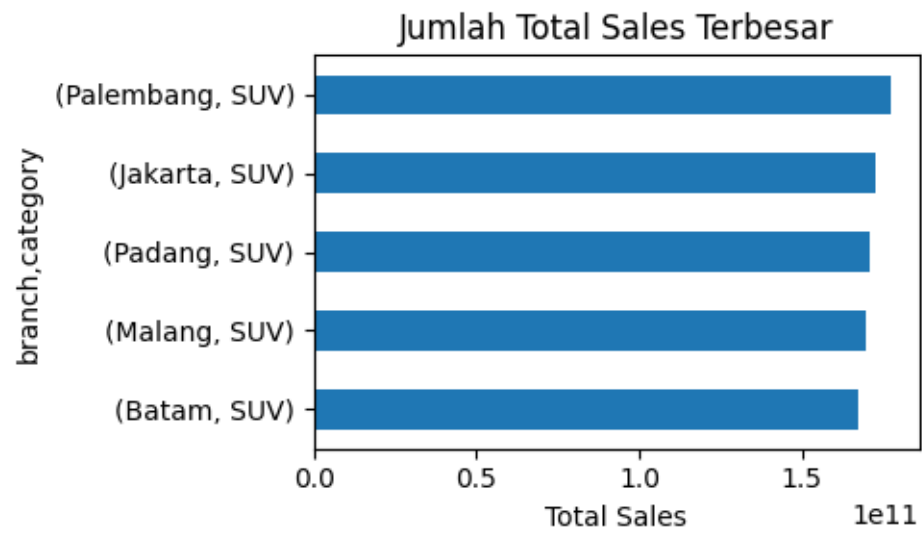
# caranya sama seperti bar normal
# yang membedakan horizontal menggunakan barh
# total sales dipersingkat secara default dikarenakan nilai terlalu besar
# keterangan 1e11 = 1x10^11

# Chart total_sales_tertinggi
total_sales_tertinggi.head(5).sort_values().plot(
    kind="barh",
    figsize=(5, 3),
    title="Jumlah Total Sales Terbesar"
)
plt.xlabel("Total Sales")
plt.tight_layout()
plt.show()

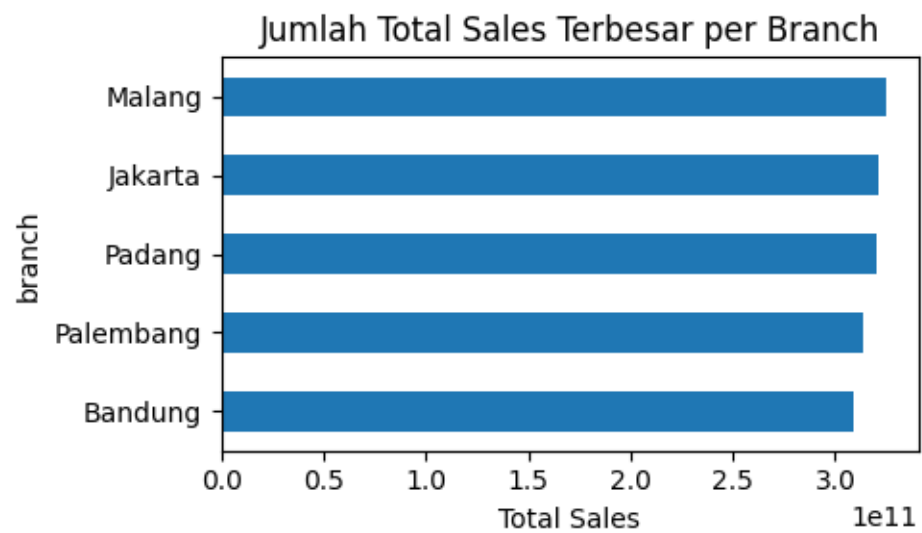
# Chart branch_tertinggi
branch_tertinggi.head(5).sort_values().plot(
    kind="barh",
    figsize=(5, 3),
    title="Jumlah Total Sales Terbesar per Branch"
)
plt.xlabel("Total Sales")
plt.tight_layout()
plt.show()

# Chart category_tertinggi
category_tertinggi.head(5).sort_values().plot(
    kind="barh",
    figsize=(5, 3),
    title="Jumlah Total Sales terbesar per Kategori"
)
plt.xlabel("Total Sales")
plt.tight_layout()
plt.show()
```

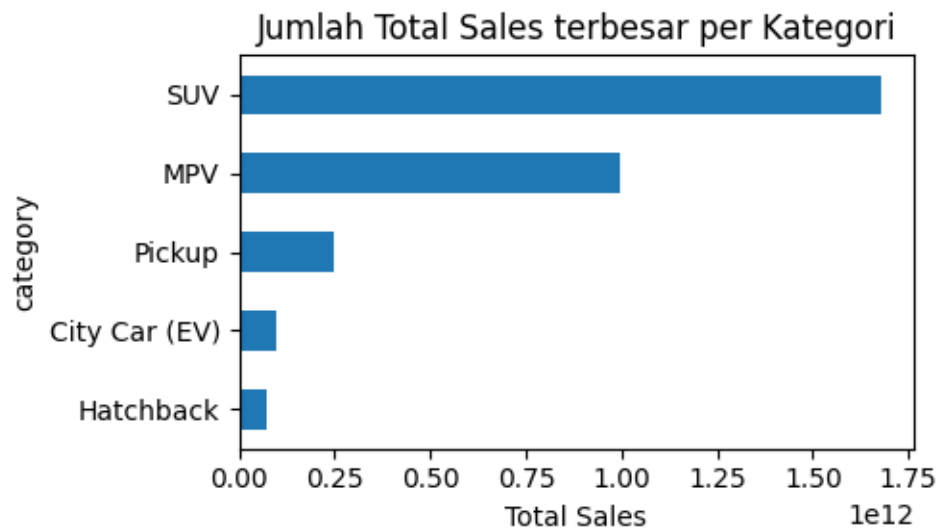
Output Kombinasi



Output Branch



Output Category



5. Optional. Code saya tampilkan dibawah

```
# # (opsional) Tantangan tambahan: rata-rata harga per unit untuk 5 kombinasi teratas
# # karena yang dibutuhkan kombinasi jadi yang saya tulis hanya satu tidak seperti soal diatas

# Mengambil total sales dan total quantity
avg_kombinasi = (
    df.groupby(["branch", "category"])
    .agg(
        total_sales = ('total_sales', 'sum'),
        total_quantity = ('quantity', 'sum')
    )
)

# Menghitung rata-rata
avg_kombinasi["avg_sales"] = (
    avg_kombinasi["total_sales"] / avg_kombinasi["total_quantity"]
)

# Mengurutkan data
avg_kombinasi = avg_kombinasi.sort_values("avg_sales", ascending=False)

# Menampilkan data
avg_kombinasi.head(5)
```

Output

		total_sales	total_quantity	avg_sales	
branch	category				
Kendari	Pickup	30209000000	64	4.720156e+08	
Palembang	Pickup	24338000000	52	4.680385e+08	
Batam	Pickup	27980000000	60	4.663333e+08	
Padang	Pickup	20002000000	43	4.651628e+08	
Jakarta	Pickup	26885000000	58	4.635345e+08	